Pruebas automatizadas del gestor de tareas Estudio

Se ha generado y ejecutado un entorno de pruebas sobre la aplicación de la actividad 1. El resultado final es de 42 métodos correctos: 33 de validación, almacenamiento y API y 9 de interfaz. El proceso detectó y corrigió un defecto de cierre de conexiones SQLite.

Objetivo y alcance

El objetivo es comprobar los flujos esenciales y las entradas que podrían comprometer los datos. Se usan SQLite y HTTP reales para integración, y Microsoft Edge para ejecutar la interfaz. Todo el código de pruebas y la configuración del entorno fueron generados por el asistente.

Prioridad	Riesgo	Validación prevista
Alta	Pérdida o corrupción de tareas	CRUD, persistencia y edición inválida
Alta	Transiciones incorrectas	Completar, reabrir y cancelar borrado
Alta	Entradas inválidas o código	Tipos, límites, fechas, SQL y HTML
Alta	Cálculo de tareas vencidas	Ayer, hoy, mañana, sin fecha y completada
Media	Búsqueda y filtros incorrectos	Combinación y resumen global
Media	Problemas de interfaz	Móvil, teclado y error de conexión

Criterio de finalización

Ambas suites deben terminar sin fallos, errores ni omisiones, y deben conservarse logs y capturas. La cobertura se expresa por funcionalidades y casos; no se ha medido un porcentaje de cobertura de líneas.

Entorno reproducible y aislamiento

Las pruebas unitarias y de integración utilizan unittest, sqlite3 y urllib de la biblioteca estándar. Las pruebas de interfaz utilizan Playwright 1.64.1 con el canal msedge.

Elemento	Entorno utilizado
Sistema	Windows y ejecución local
Python	3.12.14
Framework base	unittest incluido en Python
Interfaz	Playwright 1.64.1
Edge	156.0.4314.58
Reloj del navegador	Europe/Madrid; idioma es-ES

Ejecución de validación e integración

```
cd estudio
python run_tests.py
```

Ejecución de interfaz en Windows

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.venv\Scripts\python.exe tests/browser_checks.py
```

Se requiere Microsoft Edge instalado. Los ejecutores devuelven código 0 al pasar y 1 al fallar. run_tests.py escribe pruebas.log y resultado.json; browser_checks.py escribe interfaz.log, resultado_interfaz.json y las capturas.

Aislamiento de datos y procesos

Cada prueba de repositorio crea un directorio temporal. La suite de API utiliza otro archivo SQLite y un puerto asignado por el sistema. Cada método de interfaz dispone de una base temporal, un servidor y un contexto de navegador propios. El cierre de conexiones, páginas y servidores permite limpiar los archivos sin afectar la base del usuario.

Casos y trazabilidad

Los identificadores RF01 a RF09 proceden de la definición del proyecto. Los nombres completos de los métodos se encuentran en los logs y en los dos archivos de pruebas.

Requisito	Casos implementados	Evidencia
RF01 y RF03	Título vacío o espacios; tipos; límites; fechas; edición inválida sin cambios	ValidationTests y StoreTests
RF02	Nueva instancia de repositorio y reinicio real del servidor	test_persists_in_new_store y UI 05
RF03 a RF05	CRUD HTTP; estados; completar y reabrir; cancelar y confirmar borrado	test_crud_lifecycle y UI 01
RF06 y RF07	Búsqueda en tres campos; mayúsculas; filtros combinados	StoreTests, API y UI 02
RF08	Resumen global independiente de la lista filtrada	UI 02 y UI 09
RF09	Fechas ayer, hoy y mañana; sin fecha; completada; combinación	UI 09
RNF	Entrada SQL literal; HTML como texto; origen ajeno; móvil; Escape; red	StoreTests, API y UI 04, 06, 07, 08

Resultados esperados relevantes

Una edición inválida debe rechazar la petición y conservar el registro original. Un identificador inexistente debe producir 404. El borrado confirmado debe devolver 204. Una petición con JSON no válido debe producir 400; un origen ajeno, 403; y un tipo de contenido incorrecto, 415.

La prueba del filtro vencido crea seis tareas: solo las dos pendientes con fecha de ayer se muestran inicialmente. Al combinar búsqueda, prioridad y estado queda una. Las tareas completadas, de hoy, de mañana y sin fecha quedan fuera de esa selección.

El total cuenta métodos de prueba; los bucles de subcasos y las múltiples aserciones no se contabilizan como pruebas adicionales.

Fallo encontrado y corrección

La primera ejecución reveló un defecto de gestión de recursos. Los métodos de almacenamiento alcanzaban sus comprobaciones, pero Windows impedía borrar las bases temporales durante tearDown porque permanecían conexiones abiertas.

Ejecución	Métodos	Fallos	Errores	Resultado
Inicial de lógica y API	33	0	12	No correcta
Tras cerrar conexiones	33	0	0	Correcta
Interfaz final	9	0	0	Correcta

Extracto del error real

```
PermissionError: [WinError 32]
... el archivo está siendo utilizado por otro proceso
... tasks.db
Ran 33 tests ...
FAILED (errors=12)
```

Los 12 errores comparten causa: with sqlite3.connect gestiona la transacción, pero el diseño inicial no cerraba explícitamente la conexión. La corrección añadió un gestor de contexto que asegura close en finally después de la transacción.

```
@contextmanager
def connect(self):
    db = sqlite3.connect(self.path)
    db.row_factory = sqlite3.Row
    try:
        with db:
            yield db
    finally:
        db.close()
```

La siguiente ejecución de las 33 pruebas pasó. Se conservan primera_ejecucion.log y primer_resultado.json, además de los resultados correctos posteriores. No se ocultó el fallo ni se retiraron las pruebas que lo detectaron.

Comprobación de regresión

Tras la solicitud del filtro fuera de plazo se añadió UI 09 y se repitió completa la suite de navegador. Los nueve métodos pasaron. La mejora afectó a la selección de tareas en la interfaz y no modificó la API ni el esquema de base de datos.

Evidencias de ejecución

Las capturas se obtuvieron de la aplicación ejecutándose. El filtro fuera de plazo deja una tarea visible en la base de ejemplo, mientras el resumen mantiene los totales generales.

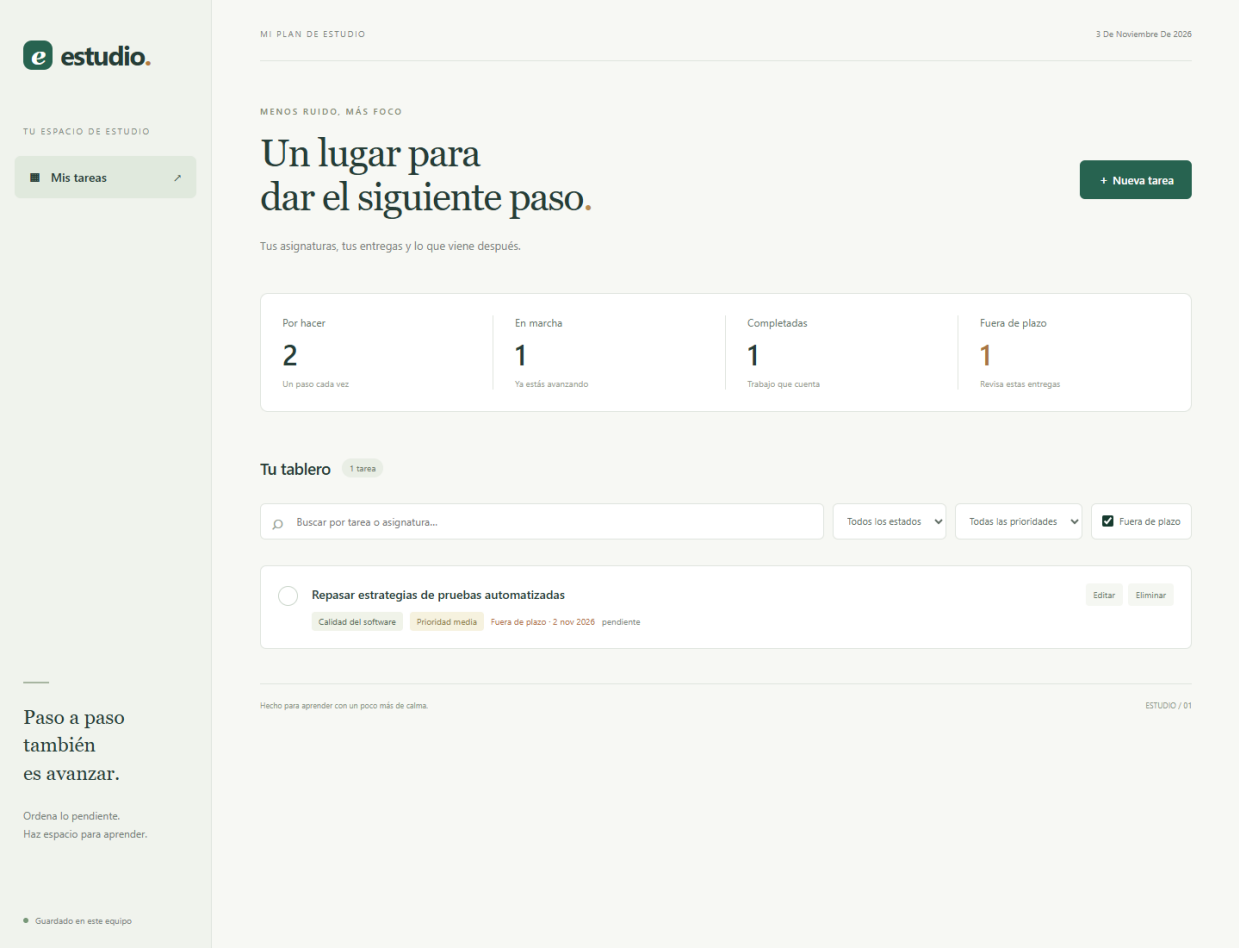


Figura 1. Filtro solicitado por el usuario en funcionamiento. Archivo: evidencias/05_fuera_de_plazo.png.

Extractos de los logs finales

```
----------------------------------------------------------------------
Ran 33 tests in 1.234s

OK

----------------------------------------------------------------------
Ran 9 tests in 13.524s

OK
```

Ejecución de lógica y API: 2026-11-03T19:07:14.934733+00:00 UTC. Ejecución de interfaz: 2026-11-03T19:10:01.054807+00:00 UTC. Los archivos JSON conservan las marcas temporales completas.

Se adjuntan los logs completos y una captura de su visualización en evidencias/04_resultados.png. Las capturas adicionales muestran escritorio, formulario de edición y móvil.

Interacción y análisis crítico

La interacción partió de la aplicación desarrollada en la actividad 1. El usuario eligió el proyecto y solicitó una mejora tras ver el prototipo; el asistente generó el entorno, las pruebas y la corrección del defecto detectado.

Extractos reales de la conversación

Usuario: «Gestor de tareas de estudio con base de datos local (recomendado)».

Asistente: «Después añadiré pruebas unitarias y de integración, comprobaré la interfaz y prepararé los dos informes en PDF».

Asistente, tras la primera ejecución: «Las conexiones de SQLite quedaban abiertas y Windows impedía borrar las bases de datos temporales. Ya he corregido el cierre de las conexiones».

Usuario, tras ver el tablero: «Añadir un filtro de tareas fuera de plazo».

Asistente: «Añadiré una prueba para comprobar que excluye las tareas completadas, las que vencen hoy y las que no tienen fecha».

Estas intervenciones se conservan en documentacion/interacciones.md. No hubo un prompt técnico de pruebas redactado por el usuario: la propuesta del entorno fue del asistente. La petición de mejora sí produjo un criterio concreto y una prueba nueva.

Interpretación de los resultados

Las pruebas validan los flujos críticos y muestran que la automatización permitió detectar un defecto que no era visible al usar el tablero. La limpieza de temporales comprobó indirectamente la liberación de recursos. El caso RF09 introdujo límites de fecha y evitó equiparar cualquier tarea antigua con una tarea pendiente vencida.

Límites y mejoras

No se midió cobertura de líneas ni se aplicaron pruebas de mutación. No se probaron carga, concurrencia sostenida, varias versiones de navegador, accesibilidad completa ni recuperación de una base corrupta. El caso de fallo de red verifica que aparece un mensaje, pero no compara su traducción. El entorno ejecutado fue Windows con Edge; otros sistemas no están verificados.

Como siguiente mejora conviene ampliar la revisión humana de los casos y evaluar accesibilidad y recuperación de datos. La calidad no debe deducirse solo de un resultado verde: también depende de que las reglas y los resultados esperados expresen lo que necesita el usuario.

Fuentes de trabajo: enunciado psuiadesoft02_act2.docx; tests/test_app.py; tests/browser_checks.py; evidencias de ejecución y extractos reales del chat del 3 de noviembre de 2026.